

Stretch challenges

For trainees who finish the core notebook early. Four tasks: the first three are the core experiments; the fourth is an optional advanced extension. Instructor solutions are in **answer-key.pdf** — try each task before peeking.

Each task is a small change to `rag_fundamentals_lab.ipynb`. The pipeline is already parameterised, so most are a one-line edit followed by **Restart & Run All** (or just re-running the affected cells). For every task, **predict the result first, then run it, then explain any surprise**.

Stretch 1 — Top-K experiment

Goal: understand how the number of retrieved chunks affects the evidence (and the answer).

Do this. Section 7 / the stretch cell already loops over `top_k`. Extend it to compare the *answers*, not just the sources:

```
question = "Which flag did lumen sync add in version 2.3?"
for k in (1, 3, 5):
    result = generate_answer(question, top_k=k)
    print(f"\n--- top_k = {k} ---")
    print(format_with_citations(result))
```

Then try a second question that genuinely needs only one chunk (e.g. the HTTP 404 question) and a vaguer one (e.g. “How do I organise my code?”).

Questions to answer. 1. As `top_k` grows, what happens to the list of sources? Do the *new* chunks help? 2. Does a larger `top_k` ever change the actual answer, or just pad it? 3. When would a **larger** `top_k` genuinely be better? When would it hurt?

Stretch 2 — Chunk size experiment

Goal: feel the chunk-size trade-off (the “U-curve”) for yourself.

Do this. Re-chunk and re-embed the corpus at three sizes, then retrieve the same query against each and compare. For *this* experiment, chunk the **raw document text** directly (not the per-section blocks the core notebook uses) so that *chunk size* — not the headings — controls the pieces. Add a cell:

```
query = "How do you define a function in Python?"
raw = load_documents(KB_DIR)
for size, overlap in [(15, 4), (60, 15), (400, 40)]:           # small / medium / large
    chunks = [
        {"id": f"{d['source']}#{i}", "text": piece,
         "source": d["source"], "section": None}
        for d in raw
        for i, piece in enumerate(chunk_text(d["text"], size, overlap))
    ]
    chunk_vectors = embed_texts([c["text"] for c in chunks]) # reassigns the index
    top = retrieve(query, top_k=1)[0]
    print(f"size={size:>3}w -> {len(chunks):2d} chunks | top score {top['score']:.3f} "
          f"| {len(top['chunk']['text'].split())} words in top chunk")
```

(Note: `retrieve` reads the module-level `chunks/chunk_vectors`, so reassigning them at the top of the loop is exactly what re-points the index. Re-run the core Section 4 cell afterwards to restore the section-aware index.)

Questions to answer. 1. How does the **number of chunks** change with size? Why? 2. At the **smallest** size, is the retrieved chunk still self-contained and useful? 3. At the **largest** size, what is now *inside* the single retrieved chunk besides the answer? How might that hurt a real answer? 4. Where is the sweet spot for *this* corpus, and why is “it depends” the honest general answer?

Stretch 3 — Grounding guard (refuse on weak evidence)

Goal: make the system refuse when nothing is relevant enough — and see how the threshold controls the trade-off between answering and refusing.

Do this. `generate_answer` already takes `score_threshold`. Sweep it against a question that *should* be answered and one that *should* be refused:

```
answerable = "What does HTTP status code 404 mean?"
unanswerable = "What is the capital of Mars?"
for threshold in (0.0, 0.1, 0.2, 0.4, 0.8):
    a = generate_answer(answerable, score_threshold=threshold)["refused"]
    u = generate_answer(unanswerable, score_threshold=threshold)["refused"]
    print(f"threshold={threshold:>3}: answerable refused? {a:<5} "
          f"unanswerable refused? {u}")
```

Questions to answer. 1. At threshold = 0.0, is the Mars question refused? Why is a zero threshold unsafe? 2. Find a threshold that answers the 404 question **and** refuses the Mars question. How wide is the safe range for this corpus? 3. If you set the threshold **too high**, what failure appears — and why is it sometimes the *safer* failure in a high-stakes domain? 4. Why can't a single magic threshold be copied between two different corpora or embedding models?

Stretch 4 (optional, advanced) — Embeddings bake-off: fallback vs MiniLM

Goal: see how embedding *quality* changes retrieval — and why the backend matters.

Requires internet once to `pip install sentence-transformers` and download `all-MiniLM-L6-v2` (~90 MB). Skip if you're offline.

Do this. If your Section 5 banner says “offline fallback,” install the model and re-run so the banner shows `sentence-transformers / all-MiniLM-L6-v2`. Keep a note of the retrieval scores for these queries under **each** backend:

```
for q in ["How do I declare a procedure in Python?",          # no shared keyword with the source
          "Which flag did lumen sync add in version 2.3?",
          "What is the capital of Mars?"]:
    hits = retrieve(q, top_k=1)
    print(f"{hits[0]['score']:.3f} {hits[0]['chunk']['section']:<28} <- {q}")
```

Questions to answer. 1. The first query says “**declare a procedure**,” which shares almost no words with “define a function.” Which backend still retrieves the right chunk, and why? 2. Do the raw score *numbers* mean the same thing across backends? What does that imply for reusing a `score_threshold` (link back to Stretch 3)? 3. Which misconception from `misconceptions.pdf` does this experiment make concrete?